

Time Stamp

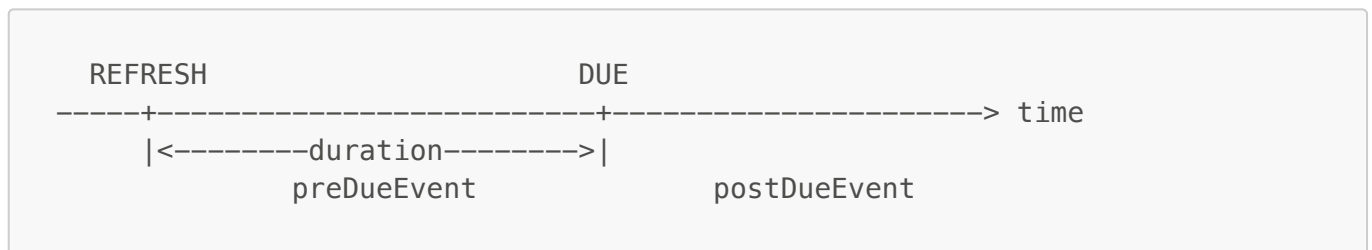
Author: Zihui(Andy) Liu <liuzihui@uw.edu, salzhx@gmail.com> Last Update: Jan 4, 2021

Overview

This is the library that helps record time in Arduino Environment. Additionally, users can register time-sensitive events and automatically invoke them when checking status. Users can set up many exciting behaviors like periodic timers using this macro library.

Event Timeline

- The `duration` (unit: milliseconds) field sets a boundary between `preDueEvent` and `postDueEvent`. The due time is refresh time + duration. The illustration of the timeline is below



Interfaces

Time Stamp Initialization

- `TS_CREATE(tsName, duration)`
 - Initialize Time Stamp with given name `tsName`
 - Create necessary variables
 - `TS_tsName_timeStamp` stores the time when last `TS_REFRESH` is called. Its value is set to current time.
 - `TS_tsName_duration` stores the duration between the last time stamp and due (see diagram in *Event Timeline* section).
 - Create time stamp with this macro if it is not intended to assign any events.
- `TS_CREATE_W_EVENT(tsName, duration)`
 - This macro is the combination of `TS_CREATE` and `TS_DUE_EVENT_DECLARATION`
 - Create necessary variable and declare necessary functions.
 - `TS_DUE_EVENT_DEFINITION` must be defined the source file.

Time Stamp Control

- `TS_REFRESH(tsName)`
 - Refresh the time stamp variable `TS_tsName_timeStamp`, which equivalent to reset timer.
- `TS_IS_DUE_PASSED(tsName)`

- Check if the due is passed.
- Return `true` if the due is passed. Otherwise, return `false`.
- `TS_SET_DURATION(tsName, duration)`
 - Change the duration settings, i.e. change `TS_tsName_duration` field.
- `TS_TIME_ELAPSED_MS(tsName)`
 - Return the time elapsed since last refresh

Time Stamp Event

- `TS_DUE_EVENT(tsName)`
 - Invoking the event based on the current time.
 - See diagram in *Event Timeline* section.
 - `TS_PRE_DUE_EVENT(tsName)`: This function will be invoked when calling `TS_DUE_EVENT()` while before due.
 - `TS_POST_DUE_EVENT(tsName)`: same as above but after due.
- `TS_DUE_EVENT_DECLARATION(tsName)`
 - Declare functions headers for invoking events.
 - `TS_tsName_preDueEvent()` and `TS_tsName_postDueEvent()`.
 - If timeStamp is created with the `TS_CREATE_W_EVENT` macro, do not call this macro.
- `TS_DUE_EVENT_DEFINITION(tsName, preDueEvent, postDueEvent)`
 - Define events both before and after the event.
 - `preDueEvent` and `postDueEvent` should contains regular C/C++ code. Both field can be left empty or assign `TS_NO_OP` to indicate no event.
- `TS_NO_OP`
 - No operation
 - Can be used in `TS_DUE_EVENT_DEFINITION()` as a placeholder for events.
- `TS_PRE_DUE_EVENT(tsName)`
 - Invoking the pre due event without checking due.
- `TS_POST_DUE_EVENT(tsName)`
 - Invoking the post due event without checking due.

Examples

- The following code demonstrate basic time stamp functions
 - The built-in LED (pin 13) will lit up after one second without any button input (pin 12)
 - Pressing the button (set pin 12 to low/GND) reset the timeStamp.

```
#include <Arduino.h>
#include "TimeStampMacro.h"

// Create timeStamp named "ts1" with due event functionality.
// Set its duration for 1000 ms, or 1 second.
TS_CREATE_W_EVENT(ts1, 1000)

void setup() {
  // initialization
  Serial.begin(9600);
```

```
pinMode(13, OUTPUT);
pinMode(12, INPUT_PULLUP);
TS_REFRESH(ts1);           // Reset timeStamp right before program
started
}

void loop() {
    TS_DUE_EVENT(ts1)       // Invoking the event

    if (!digitalRead(12)) { // If button is pressed, i.e. pin 12 is low
        TS_REFRESH(ts1);    // reset timeStamp
    }

    delay(200);             // "clock speed"
}

TS_DUE_EVENT_DEFINITION(ts1,
    // preDueEvent
    Serial.println("preDue");
    digitalWrite(13, LOW);
    ,
    // postDueEvent
    Serial.println("postDue");
    digitalWrite(13, HIGH);
)
```

Note

This timer library is passive, i.e., `preDueEvent` and `postDueEvent` will be invoked by themselves automatically. Users need to actively call `TS_DUE_EVENT(tsName)` to invoke corresponding events.

Dependency

- `Arduino.h`, only `millis()` is needed
- The library is developed with C++11 preprocessor and tested using C++ compiler. However, it *should* be compatible with C (NOT TESTED).

V2

Simplified version with commonly used functions

```
TIMER_CREATE(tsName)

TIMER_RESET(tsName)
TIMER_TIME_ELAPSED(tsName)
TIMER_IS_TIME_ELAPSED(tsName, time_elapsed)
```

Update History

- Oct 10, 2021
 - Added basic functionalities Time Stamp
- Jan 4, 2022
 - Added `TS_TIME_ELAPSED_MS(tsName)` function
- Oct 10, 2025
 - Added V2